

FACULDADE DE TECNOLOGIA PADRE DANILO JOSÉ DE OLIVEIRA OHL

FATEC BARUERI

Gabriel de Souza Rosado

Igor Akitomi Emygidio

Jefferson Luiz da Silva Ferreira

Leonardo de Novaes Vivi

Vinícius da Silva Ferreira

**DESENVOLVIMENTO DE UMA SOLUÇÃO WEB PARA DIVULGAÇÃO E GESTÃO
DE SERVIÇOS DE ASSISTÊNCIA TÉCNICA**

Desenvolvimento de sistema para controle de O.S, vendas e estoque com
arquitetura moderna.

BARUERI

2026

FACULDADE DE TECNOLOGIA PADRE DANILO JOSÉ DE OLIVEIRA OHL

FATEC BARUERI

Gabriel de Souza Rosado

Igor Akitomi Emygidio

Jefferson Luiz da Silva Ferreira

Leonardo de Novaes Vivi

Vinícius da Silva Ferreira

**DESENVOLVIMENTO DE UMA SOLUÇÃO WEB PARA DIVULGAÇÃO E GESTÃO
DE SERVIÇOS DE ASSISTÊNCIA TÉCNICA**

Desenvolvimento de sistema para controle de O.S, vendas e estoque com
arquitetura moderna.

Trabalho de Conclusão de Disciplina
apresentado ao curso de Tecnologia da
Informação da Faculdade de Tecnologia
Padre Danilo de Oliveira OHL, como
requisito parcial para obtenção do título de
graduação.

BARUERI

2026

FICHA CATALOGRÁFICA

Autor: Gabriel de Souza Rosado

Título: Criação de um site para uma loja de assistência técnica focada em eletrônicos: TrinixTech / Gabriel de Souza — Barueri, 2026.

Orientador: Vander Ribeiro Elme

Projeto Interdisciplinar AAP (Tecnologia em Gestão da Tecnologia da Informação) — Faculdade de Tecnologia de Barueri — FATEC Barueri, 2026.

1. Tecnologia da Informação. 2. Desenvolvimento web. 3. Assistência técnica. 4. Sistemas de informação.II. Título.



Curso Superior de Tecnologia em Gestão de Tecnologia da Informação

Declaração de viabilidade prática de Projeto Interdisciplinar

Declaração de Projeto Interdisciplinar com conclusão integral com a empresa TrinixTech

Declaro, para os devidos fins, que acompanhei o Projeto Interdisciplinar IV, intitulado Desenvolvimento de uma solução web para divulgação e gestão de serviços da TrinixTech desenvolvido pelos alunos do 4º semestre do Curso Superior de Tecnologia em Gestão de Tecnologia da Informação da Fatec Barueri, em parceria com esta empresa.

Atesto que o projeto apresentado possui viabilidade prática e pode ser aplicado e utilizado como apoio às atividades da empresa.

Confirmo que o projeto foi concluído conforme a proposta desenvolvida durante o semestre em curso.

Barueri, SP, 13 de Junho 2026

__Leonardo N.vivi__

TrinixTech

49.987.078/0001-30

Estrada velha de Itapevi 3676- Barueri

+55 11 93216-4095

AGRADECIMENTOS

Gostaríamos de expressar nossa gratidão a todos que contribuíram direta ou indiretamente para a realização deste projeto.

Agradecemos aos nossos professores pelos ensinamentos, orientações e conhecimentos compartilhados ao longo do curso, que foram fundamentais para o desenvolvimento deste trabalho e para nossa evolução acadêmica.

Agradecemos também à empresa parceira que colaborou com este projeto, fornecendo informações, esclarecendo dúvidas e permitindo que tivéssemos contato com situações reais do ambiente profissional. Essa parceria foi essencial para a construção de uma solução mais próxima das necessidades do mercado.

Por fim, agradecemos a todos os integrantes do grupo GTI 212 pelo comprometimento, dedicação e trabalho em equipe demonstrados durante cada etapa deste desafio, tornando possível a conclusão deste trabalho com êxito.

RESUMO

Este trabalho apresenta o desenvolvimento do sistema de gestão TrinixTech que tem o objetivo otimizar e automatizar os processos administrativos da empresa. Para a execução desse projeto foram aplicados conceitos de análise e desenvolvimento de sistemas, modelagem UML, banco de dados e desenvolvimento web. Como resultado foi atingido uma solução capaz de melhorar a organização das informações, aumentar a eficiência operacional e contribuir para a tomada de decisões da empresa, facilitando o dia a dia da empresa.

Palavras-chave: Sistema de Gestão. Desenvolvimento Web. Banco de Dados. Automação. Gestão Empresarial.

RESUMO EM LINGUA ESTRANGEIRA – ABSTRACT

This paper presents the development of the TrinixTech Management System, which aims to optimize and automate the company's administrative processes. For the execution of this project, concepts of Systems Analysis and Development, UML modeling, database management, and web development were applied. As a result, a solution was achieved that can improve information organization, increasing operational efficiency, and contributing to the company's decision-making process, making its daily activities easier and more efficient.

Keywords: Management System. Web Development. Database. Automation. Business Management.

Sumário

1 INTRODUÇÃO	11
2 REQUISITOS DO SISTEMA	12
2.1 Requisitos Funcionais	12
2.2 Requisitos Não Funcionais	12
2.3 Regras de Negócio	12
2.4 Objetivo do Sistema	13
3 DIAGRAMAS	14
3.1 Diagrama de Caso de Uso	14
3.2 Diagrama de Atividades	15
3.3 Diagrama de Classes	16
3.4 Diagrama de Objetos	17
3.5 Diagrama de Componentes	18
3.6 Diagrama de Implantação	19
.....	19
3.7 Diagrama de Estado	20
Figura 7 – Diagrama de Estado.....	20
3.8 Diagrama de Colaboração	21
Figura 8 – Diagrama de Colaboração	21
4 ESCOPO DO PROJETO	22
4.1 Objetivo Geral	22
4.2 Objetivos Específicos	22
4.3 Escopo Funcional.....	22
4.3.1 Páginas a Serem Desenvolvidas	22
4.3.2 Funcionalidades.....	22
4.4 Critérios de Aceite	23
4.5 O que Está Fora do Escopo	23
4.6 Entregáveis	23
4.7 Prazo Estimado.....	23
De 3 a 6 semanas, dependendo das funcionalidades adicionais.	23
5 BANCO DE DADOS	24
5.1 Relação entre Casos de Uso e Persistência de Dados.....	24
5.2 Diagrama Entidade-Relacionamento	24
5.3 Modelo Lógico do Banco de Dados	25

5.4 Descrição das Principais Tabelas	25
5.5 Relacionamentos entre as Entidades.....	26
5.6 Regras de Integridade.....	27
5.7 Segurança, Autenticação e Controle de Acesso	27
5.8 Considerações sobre PostgreSQL e Supabase.....	27
6.1 Front-end	29
6.2 Back-end (Opcional)	29
6.3 Banco de Dados (Opcional)	29
6.4 SEO e Monitoramento.....	29
6.5 Integrações	29
6.6 Hospedagem e Deployment.....	29
6.7 Ferramentas de Suporte	30
7 FRONTEND	31
7.1 Introdução	31
7.1.1 Objetivo.....	31
7.1.2 Justificativa	31
7.2 Tecnologias usadas	32
7.3 Estrutura da aplicação	32
7.3.1 Dashboard	32
7.3.2 Ordens de serviço.....	32
7.3.3 Vendas.....	33
7.3.4 Vendas do mês	33
7.3.5 Estoque.....	33
7.3.6 Clientes	33
7.4 Interface e usabilidade	33
7.5 Considerações	34
8.1 Introdução	35
8.1.1 Objetivo.....	35
8.1.2 Justificativa	35
8.2 Tecnologias usadas	36
8.3 Modelo do banco de dados.....	36
8.3.1 Tabela de Clientes	36
8.3.2 Tabela Ordens de Serviço	37
8.3.3 Tabela Estoque.....	37

8.3.4 Tabela Vendas.....	38
8.3.5 Tabela Itens Vendas	38
8.4 Autenticação e Controle de acesso.....	38
8.4.1 Autenticação de usuários.....	38
8.4.2 Row Level Security (RLS).....	39
8.5 Integração com o Front End.....	39
8.5.1 Principais Operações.....	39
8.5.2 Baixa automática de estoque.....	39
8.6 Considerações	40
9 DOCUMENTAÇÃO DAS SPRINTS.....	41
9.1 Sprint 1 – Planejamento e Design Inicial.....	41
9.2 Sprint 2 – Design Final e Documentação.....	41
9.3 Sprint 3 – Desenvolvimento do Sistema	41
9.4 Sprint 4 – Finalização e Testes	42
10 CONCLUSÃO.....	43
11 REFERÊNCIAS.....	44

1 INTRODUÇÃO

A tecnologia está cada vez mais presente no dia a dia das empresas, deixando tarefas mais rápidas e de forma eficiente, também auxilia na organização de informações. Nessa situação, o uso de sistemas de gestão se tornou uma importante ferramenta para melhorar processos internos, reduzir erros e facilitar a tomada de decisões.

Avaliando isso, foi desenvolvido o Sistema de Gestão TrinixTech, uma plataforma criada para auxiliar no gerenciamento de produtos, serviços e vendas da empresa. O sistema tem como propósito reunir informações em um único ambiente, tornando as atividades administrativas mais organizadas e práticas.

O propósito principal deste projeto é desenvolver uma solução que contribua para a automação das demandas da empresa, possibilitando melhor gerenciamento e maior eficiência na gestão das informações. Além disso, procura-se facilitar o monitoramento das operações, melhorar a organização dos dados e oferecer mais facilidade na execução das tarefas do dia a dia.

Para a execução deste trabalho, foram utilizadas etapas de levantamento de requisitos, modelagem do sistema por meio de diagramas UML, criação do banco de dados, desenvolvimento da aplicação e realização de testes. Durante o projeto, também foram aplicados conhecimentos adquiridos ao longo do curso de Gestão da Tecnologia da Informação, permitindo a construção de uma solução estruturada às necessidades propostas.

Assim, o Sistema de Gestão TrinixTech foi desenvolvido com o propósito de contribuir para uma gestão mais organizada e eficiente, permitindo a aplicação prática dos conhecimentos adquiridos ao longo do curso.

2 REQUISITOS DO SISTEMA

2.1 Requisitos Funcionais

- ☐ RF01 – Cadastro de clientes com nome, telefone, e-mail e endereço.
- ☐ RF02 – Controle de estoque com entrada, saída e atualização automática de produtos.
- ☐ RF03 – Gerenciamento de ordens de serviço vinculadas aos clientes e equipamentos.
- ☐ RF04 – Registro de vendas com atualização automática do estoque.
- ☐ RF05 – Sistema de login para autenticação de usuários.
- ☐ RF06 – Emissão de relatórios de estoque, vendas, clientes e serviços.
- ☐ RF07 – Pesquisa de informações por nome, produto, serviço e status.

2.2 Requisitos Não Funcionais

- ☐ RNF01 – O sistema deve garantir segurança no acesso às informações.
- ☐ RNF02 – O tempo de resposta das operações deve ser rápido e eficiente.
- ☐ RNF03 – A interface deve ser simples e intuitiva para os usuários.
- ☐ RNF04 – O sistema deve funcionar em navegadores modernos e computadores com internet.
- ☐ RNF05 – O sistema deve permitir backup das informações cadastradas.
- ☐ RNF06 – O sistema deve permanecer disponível durante o horário comercial.

2.3 Regras de Negócio

- ☐ RN01 – Não é permitido realizar vendas sem produtos disponíveis em estoque.
- ☐ RN02 – Toda ordem de serviço deve estar vinculada a um cliente cadastrado.
- ☐ RN03 – Produtos com baixa quantidades devem ser identificados pelo sistema.
- ☐ RN04 – Apenas administradores podem excluir registros do sistema.

2.4 Objetivo do Sistema

O sistema foi desenvolvido para auxiliar a gestão da empresa TrinixTech, permitindo o controle de estoque, clientes, ordens de serviço e vendas, contribuindo para a organização dos processos internos e melhoria da produtividade.

3 DIAGRAMAS

Esta seção apresenta os diagramas UML elaborados para o sistema TrinixTech, abrangendo diferentes perspectivas do projeto: estrutural, comportamental e de implantação. Os diagramas foram construídos com base nos requisitos levantados e nas funcionalidades previstas para a aplicação.

3.1 Diagrama de Caso de Uso

Este diagrama apresenta as funcionalidades disponibilizadas pela landing page da TrinixTech e a interação dos usuários com essas funcionalidades. Entre elas estão visualizar informações da empresa, consultar serviços, acessar depoimentos e enviar mensagens por meio do formulário de contato.

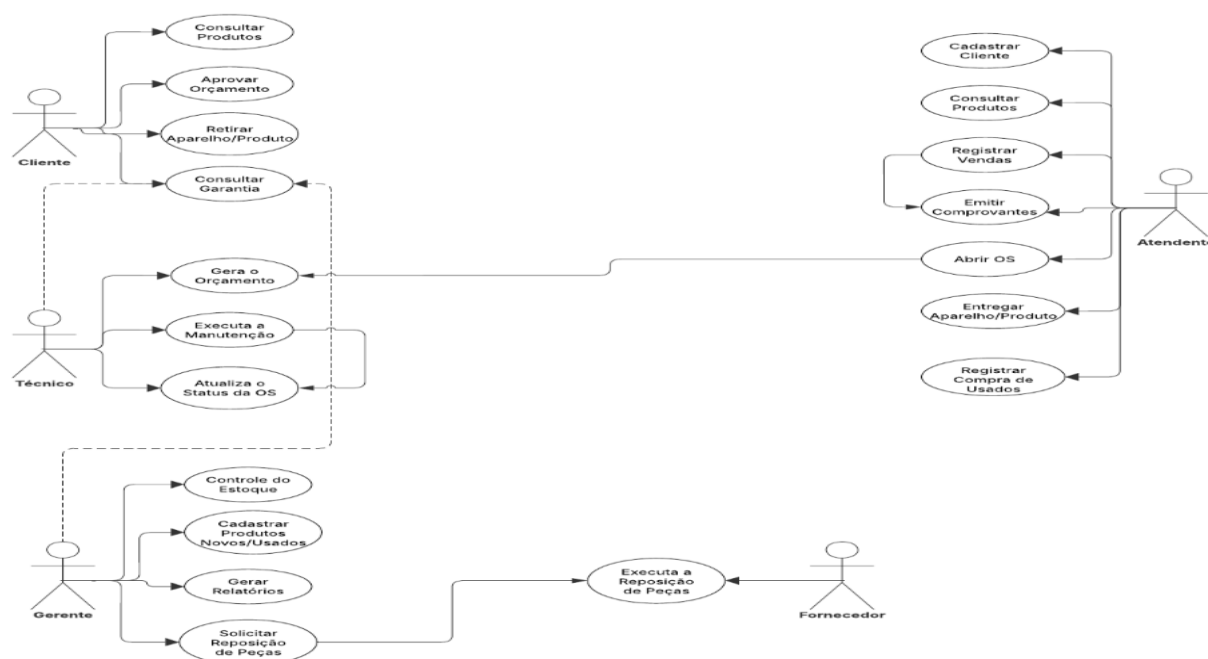


Figura 1 – Diagrama de Caso de Uso

Fonte: Elaborado pelo grupo GTI 212 (2026).

3.2 Diagrama de Atividades

Este diagrama descreve o fluxo das atividades executadas pelos usuários dentro da landing page, desde a navegação pelas seções até o envio de informações pelo formulário de contato. Ele permite visualizar o comportamento geral do sistema.

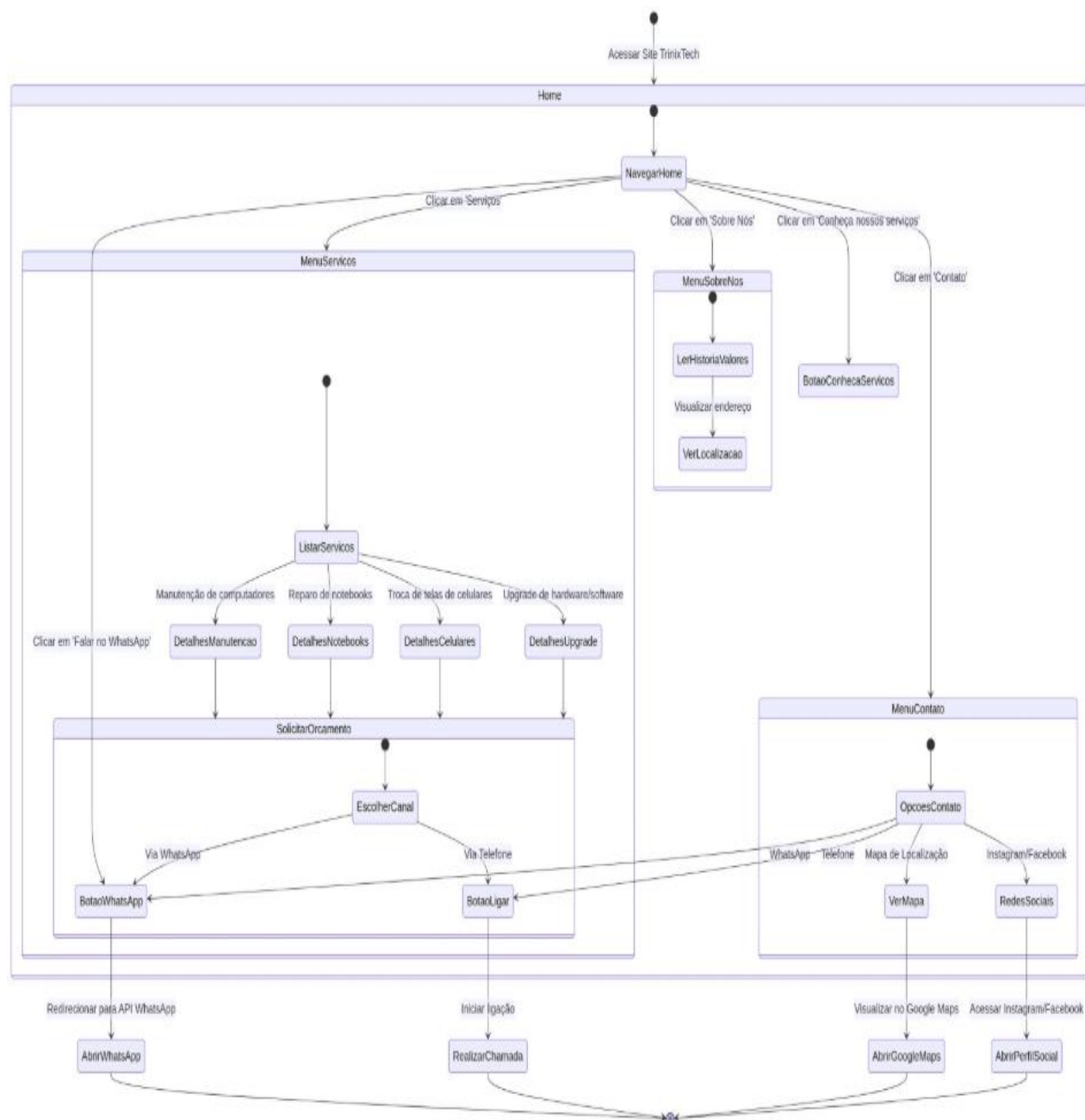


Figura 2 – Diagrama de Atividades

Fonte: Elaborado pelo grupo GTI 212 (2026).

3.3 Diagrama de Classes

Este diagrama apresenta a estrutura lógica da landing page da TrinixTech, descrevendo as classes, atributos, métodos e relacionamentos existentes. Ele serve como base para a organização e implementação do sistema.

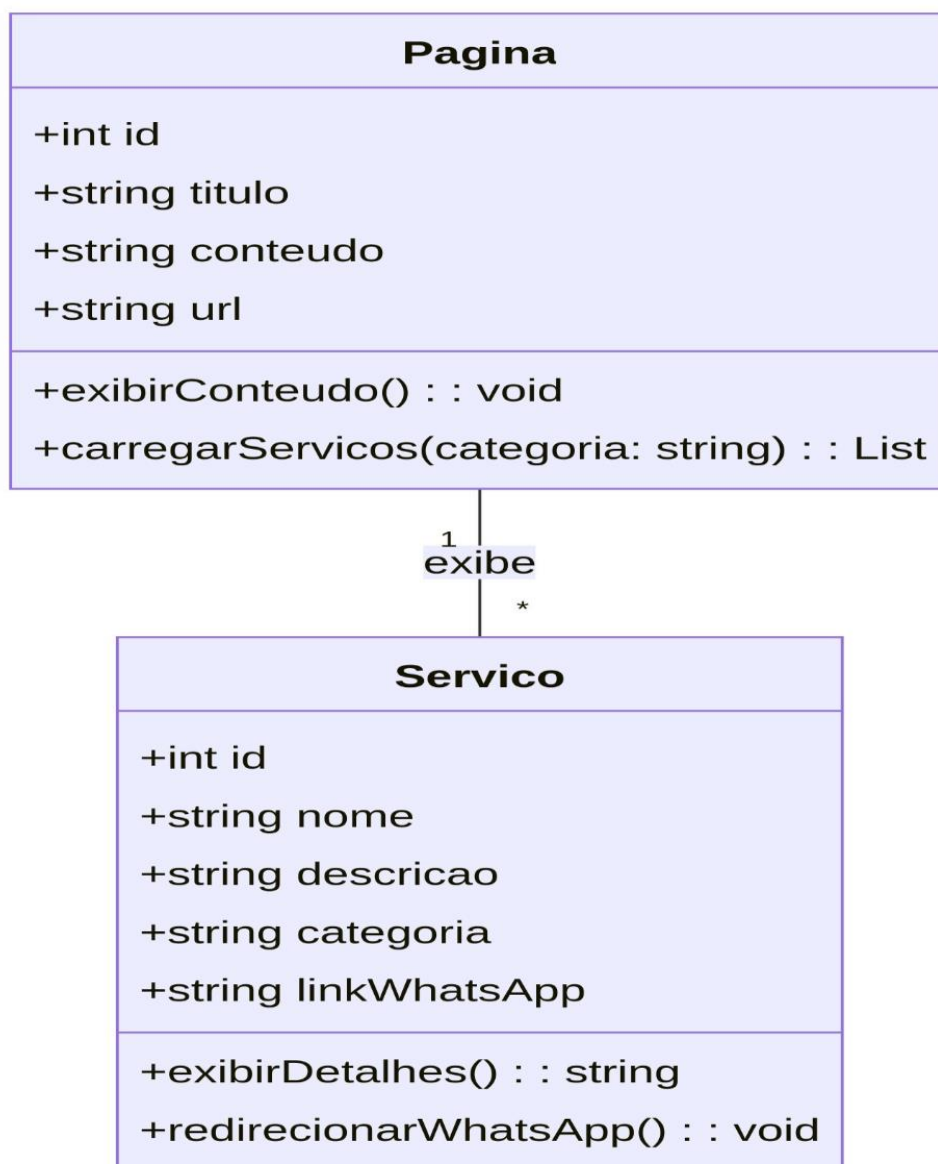


Figura 3 – Diagrama de Classes

Fonte: Elaborado pelo grupo GTI 212 (2026).

3.4 Diagrama de Objetos

Este diagrama apresenta exemplos de objetos utilizados na landing page da TrinixTech, demonstrando como as informações são armazenadas e relacionadas em tempo de execução. Entre os objetos representados estão seções da página, serviços oferecidos, depoimentos e formulários de contato.

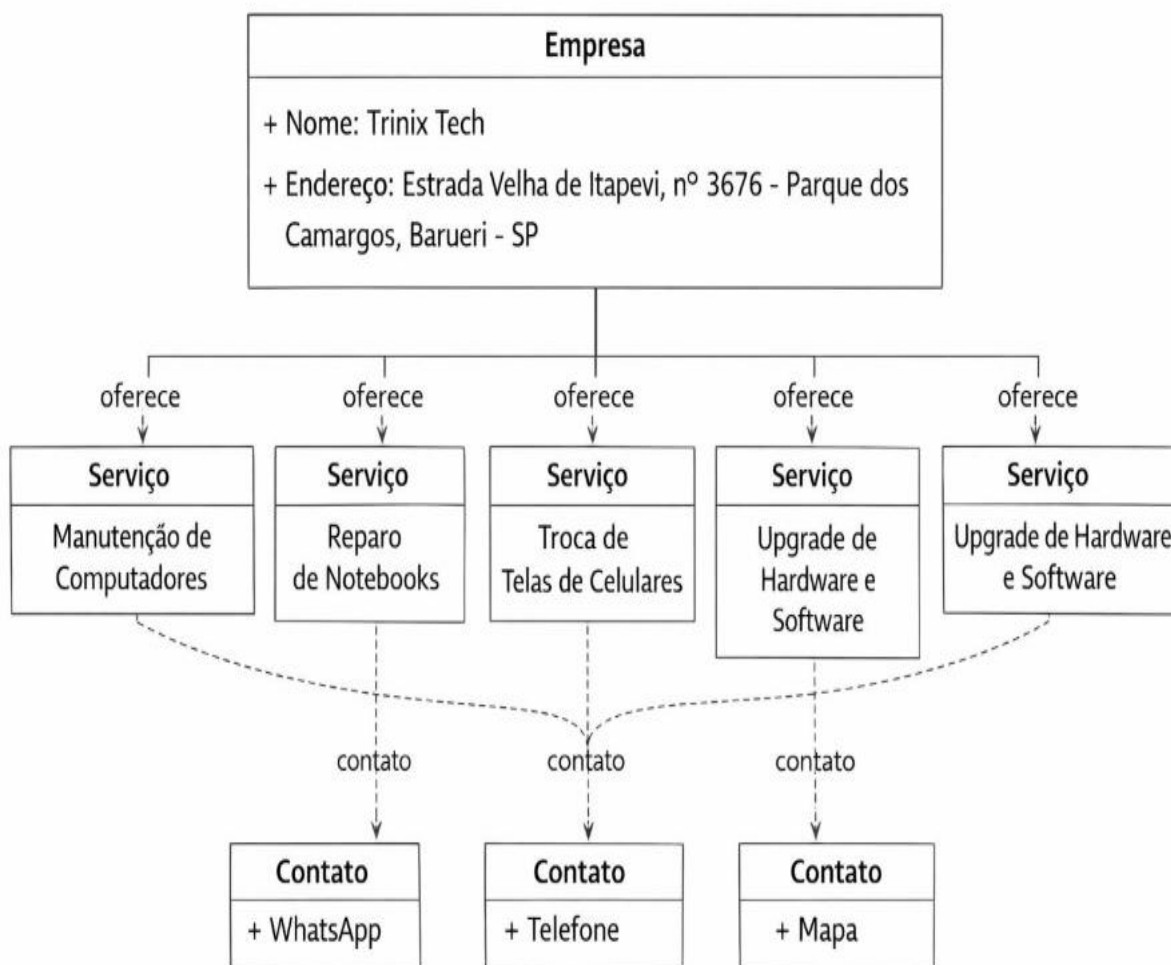


Figura 4 – Diagrama de Objetos

Fonte: Elaborado pelo grupo GTI 212 (2026).

3.5 Diagrama de Componentes

Este diagrama representa os principais componentes que compõem a landing page da TrinixTech, como interface do usuário, módulos de serviços, seção de depoimentos e formulário de contato. Também demonstra as dependências entre esses componentes.

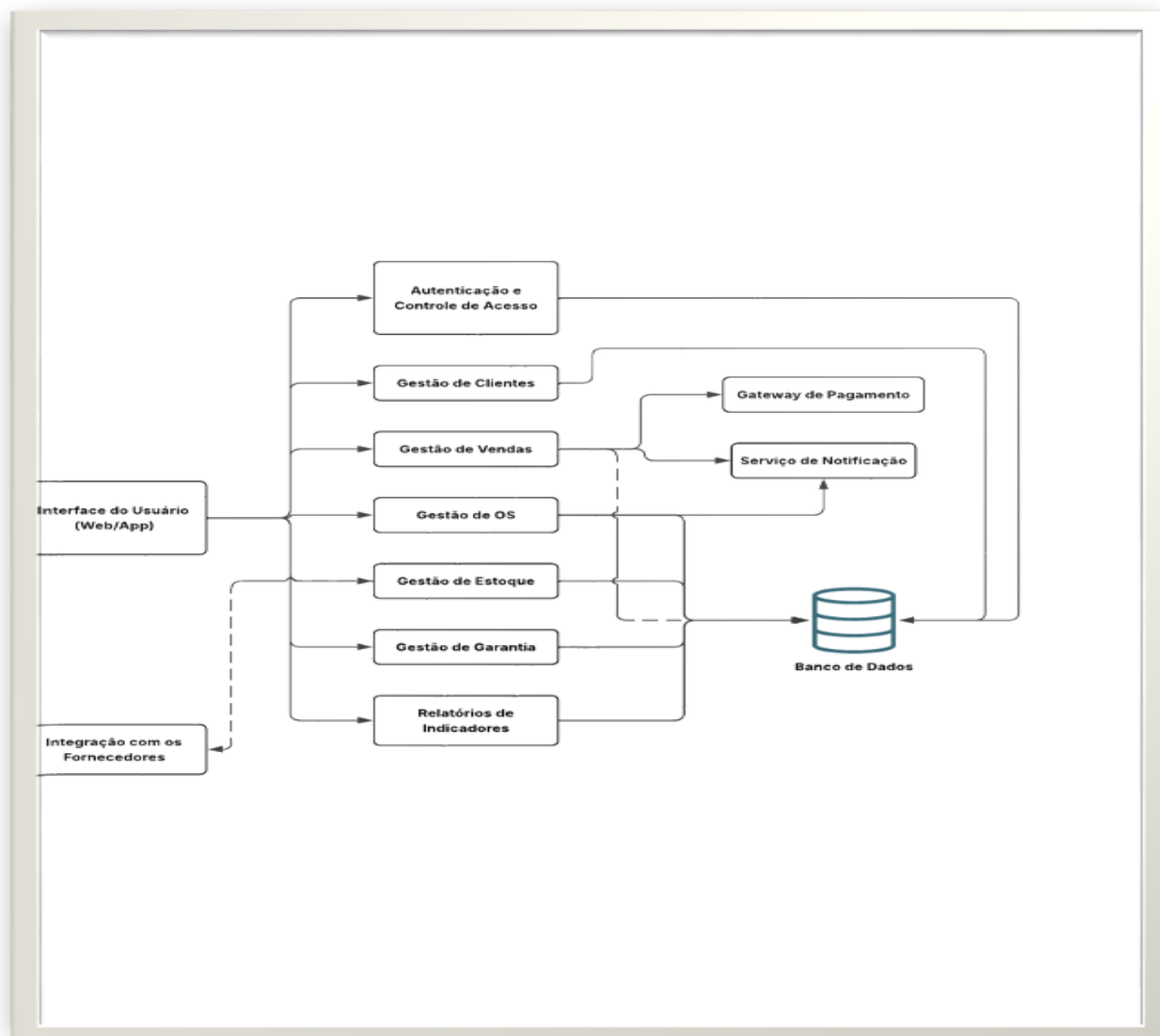


Figura 5 – Diagrama de Componentes

Fonte: Elaborado pelo grupo GTI 212 (2026)

3.6 Diagrama de Implantação

Este diagrama apresenta a estrutura física necessária para a execução da landing page da TrinixTech, incluindo servidor, banco de dados e dispositivos dos usuários. Seu objetivo é demonstrar como os componentes do sistema são distribuídos e executados no ambiente de produção.

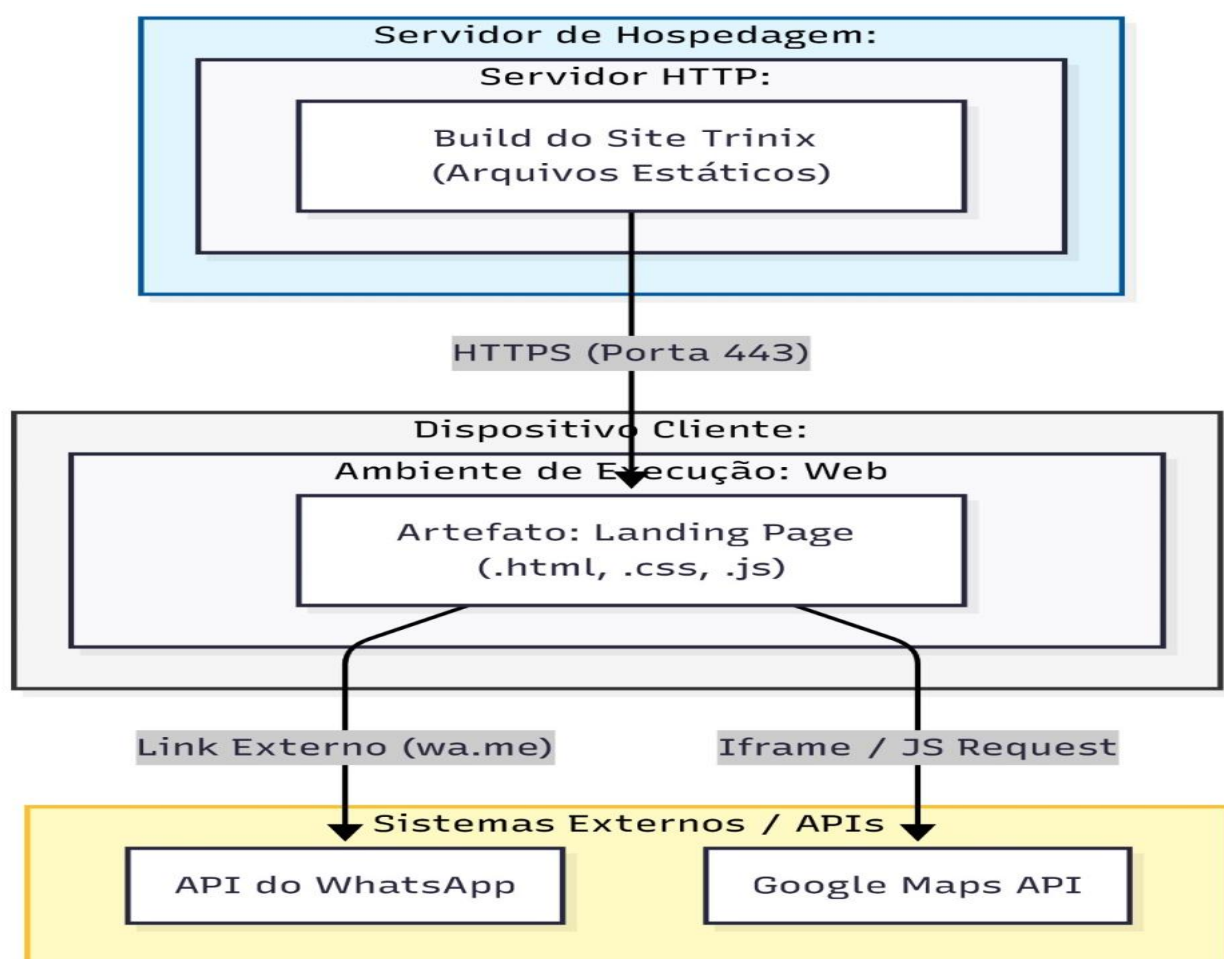


Figura 6 – Diagrama de Implantação

Fonte: Elaborado pelo grupo GTI 212 (2026).

3.7 Diagrama de Estado

Este diagrama representa os estados possíveis de determinados elementos do sistema, como formulários de contato e solicitações enviadas pelos usuários. Também demonstra as transições entre estados conforme as ações realizadas.

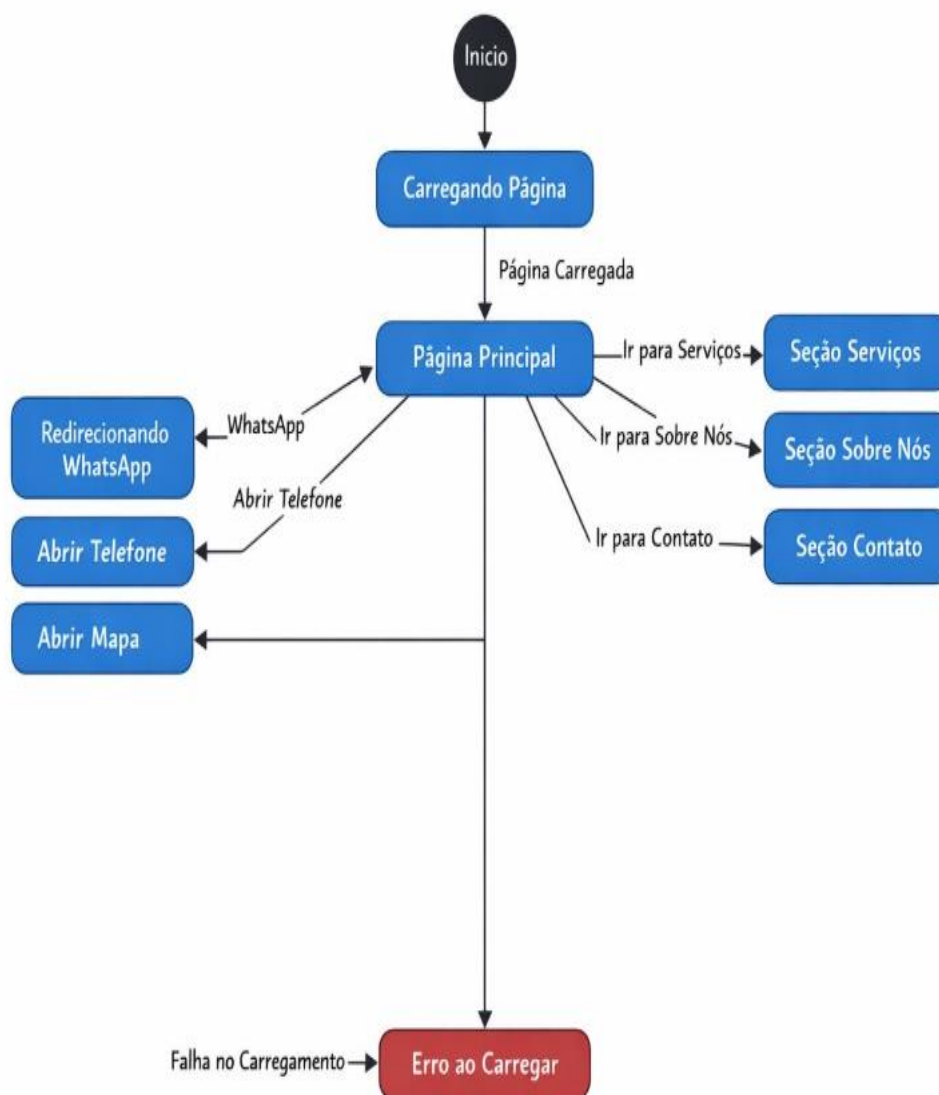


Figura 7 – Diagrama de Estado

Fonte: Elaborado pelo grupo GTI 212 (2026).

3.8 Diagrama de Colaboração

Este diagrama demonstra como os diferentes objetos e componentes da landing page colaboram entre si para executar funcionalidades específicas. Ele evidencia a comunicação necessária para garantir o funcionamento adequado do sistema.

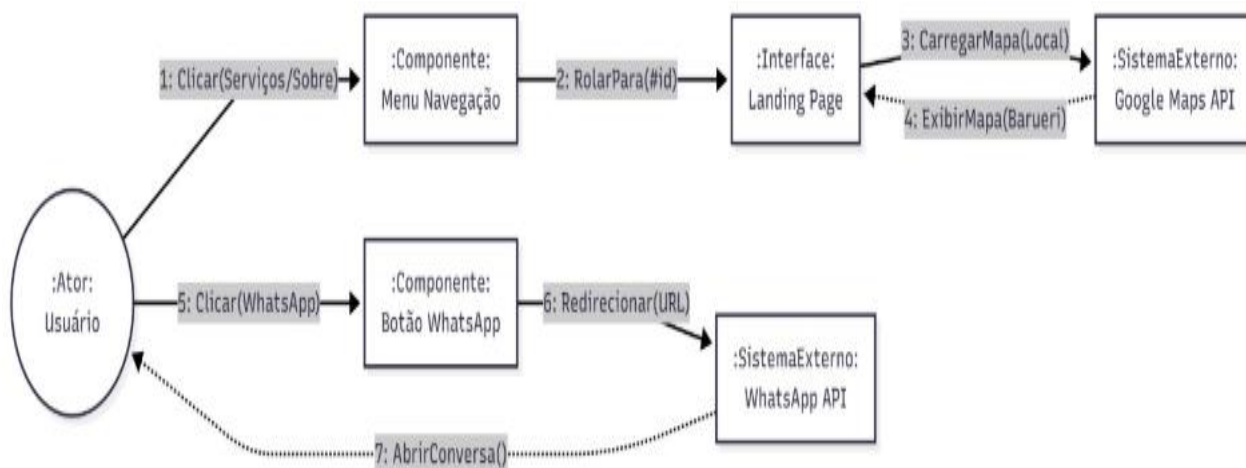


Figura 8 – Diagrama de Colaboração

Fonte: Elaborado pelo grupo GTI 212 (2026).

4 ESCOPO DO PROJETO

4.1 Objetivo Geral

Desenvolver um novo site profissional para a TrinixTech, focado em apresentar serviços, fortalecer a marca e gerar conversões através de uma interface moderna, responsiva e alinhada aos padrões atuais de UX/UI.

4.2 Objetivos Específicos

- ☐ Criar uma identidade visual consistente e alinhada com a marca.
- ☐ Desenvolver páginas institucionais e de serviços claras e estratégicas.
- ☐ Implementar formulário de contato e captação de leads.
- ☐ Otimizar o site para mecanismos de busca (SEO).
- ☐ Garantir acessibilidade e responsividade para web e mobile.

4.3 Escopo Funcional

4.3.1 Páginas a Serem Desenvolvidas

- ☐ Página inicial (Home)
- ☐ Sobre a empresa
- ☐ Serviços (com detalhamento individual)
- ☐ Portfólio / Projetos
- ☐ Contato
- ☐ Blog (opcional)

4.3.2 Funcionalidades

- ☐ Menu responsivo
- ☐ SEO básico (títulos, descrições, URLs amigáveis)
- ☐ Formulário de contato com envio para e-mail
- ☐ Integração com redes sociais

- ☐ Sessão de depoimentos (opcional)
- ☐ Área administrativa simples (opcional)

4.4 Critérios de Aceite

- ☐ Site totalmente funcional e responsivo
- ☐ Todas as páginas previstas entregues
- ☐ Testes de desempenho e compatibilidade concluídos
- ☐ Entrega de manual rápido de uso/atualização

4.5 O que Está Fora do Escopo

- ☐ Produção de textos profissionais (exceto os básicos)
- ☐ Criação de logotipo ou identidade visual completa
- ☐ Gestão de redes sociais
- ☐ Integrações avançadas não previstas inicialmente

4.6 Entregáveis

- ☐ Site completo publicado
- ☐ Arquivos fonte
- ☐ Documento com instruções de manutenção

4.7 Prazo Estimado

De 3 a 6 semanas, dependendo das funcionalidades adicionais.

5 BANCO DE DADOS

O banco de dados do sistema TrinixTech foi estruturado para atender às funcionalidades de uma aplicação web voltada à divulgação e à gestão de serviços de assistência técnica. A base de dados organiza informações relacionadas a clientes, usuários do sistema, produtos em estoque, vendas, ordens de serviço e itens utilizados em cada atendimento. Dessa forma, o banco atua como elemento central para garantir o armazenamento, a recuperação e a consistência das informações manipuladas pela aplicação.

A modelagem foi elaborada considerando os processos principais identificados no sistema, como cadastro de clientes, consulta de produtos, registro de vendas, abertura de ordens de serviço, atualização do status da manutenção, controle de estoque, geração de relatórios e solicitação de reposição de peças.

5.1 Relação entre Casos de Uso e Persistência de Dados

O diagrama de caso de uso demonstra que diferentes atores interagem com o sistema, entre eles Cliente, Atendente, Técnico, Gerente e Fornecedor. O Atendente realiza operações como cadastrar cliente, registrar vendas, emitir comprovantes e abrir ordens de serviço. O Técnico gera orçamento, executa a manutenção e atualiza o status da ordem de serviço. O Gerente realiza controle de estoque, cadastro de produtos, geração de relatórios e solicitação de reposição de peças.

A partir desses requisitos, foram definidas tabelas específicas para clientes, produtos, vendas, itens de venda, ordens de serviço, itens de ordem de serviço, perfis de usuário e papéis de acesso.

5.2 Diagrama Entidade-Relacionamento

O Diagrama Entidade-Relacionamento (DER) apresenta a visão conceitual da estrutura de dados do TrinixTech. Ele evidencia as principais entidades do sistema, seus atributos, as chaves primárias, as chaves estrangeiras e as cardinalidades existentes entre as tabelas. A modelagem considera relacionamentos do tipo um para muitos, como um cliente associado a várias vendas ou ordens de serviço, e uma venda composta por vários itens.

No DER, a entidade `customers` representa os clientes atendidos pela assistência técnica. A entidade `service_orders` representa as ordens de serviço abertas para manutenção de aparelhos. A entidade `inventory_items` armazena os produtos e peças disponíveis em estoque. Já `sales` e `sale_items` registram vendas e seus respectivos itens. Também são previstas entidades relacionadas ao controle de acesso, como `profiles` e `user_roles`.

5.3 Modelo Lógico do Banco de Dados

O modelo lógico representa a estrutura mais próxima da implementação no banco de dados relacional. Nesse nível, são definidos os nomes das tabelas, os tipos de dados, as chaves primárias, as chaves estrangeiras e os relacionamentos.

A estrutura também prevê tipos enumerados para padronizar informações sensíveis ao fluxo de negócio. O tipo `app_role` define perfis como `admin` e `atendente`,

enquanto o tipo `os_status` define estados como `aberta`, `em_andamento`, `aguardando_peca`, `pronta`, `entregue` e `cancelada`.

5.4 Descrição das Principais Tabelas

A seguir são descritas as principais tabelas que compõem a modelagem do banco de dados do sistema TrinixTech.

Tabela	Finalidade
<code>customers</code>	Armazena dados dos clientes, como nome, e-mail, telefone, observações e data de cadastro.
<code>inventory_items</code>	Registra produtos, peças e itens de estoque, incluindo categoria, SKU, preço de custo, preço de venda, quantidade disponível e estoque mínimo.
<code>service_orders</code>	Armazena as ordens de serviço abertas para manutenção, vinculando cliente, técnico, aparelho, IMEI, problema relatado, diagnóstico, status e valor total.

service_order_items	Relaciona uma ordem de serviço aos itens utilizados durante o atendimento, registrando descrição, quantidade e valor unitário.
Sales	Registra as vendas realizadas, vinculando cliente, vendedor, número da venda, valor total, forma de pagamento e data de criação.
sale_items	Armazena os itens pertencentes a uma venda, relacionando produto, quantidade e preço unitário.
Profiles	Mantém dados complementares dos usuários autenticados, como nome completo e data de criação.
user_roles	Define os papéis de acesso dos usuários do sistema, como administrador ou atendente.

5.5 Relacionamentos entre as Entidades

Os relacionamentos definidos no banco de dados permitem que os registros sejam vinculados de maneira lógica e consistente. A tabela customers relaciona-se com sales e service_orders, permitindo manter o histórico de compras e atendimentos de cada cliente. A tabela sales relaciona-se com sale_items, enquanto sale_items referencia inventory_items, possibilitando identificar quais produtos foram vendidos em cada transação.

De forma semelhante, service_orders relaciona-se com service_order_items, permitindo registrar as peças ou produtos utilizados em uma manutenção. Por fim, profiles e user_roles estão associados aos usuários autenticados, permitindo separar dados de identificação e controle de permissão.

5.6 Regras de Integridade

As regras de integridade têm a finalidade de proteger a consistência dos dados armazenados. As chaves primárias garantem a identificação única de cada registro, enquanto as chaves estrangeiras asseguram que vínculos entre tabelas só sejam realizados com registros existentes.

- ☐ Toda venda deve possuir ao menos um item associado.
- ☐ Toda ordem de serviço deve estar vinculada a um cliente cadastrado.
- ☐ A baixa de estoque deve ocorrer quando houver venda ou utilização de item em ordem de serviço.
- ☐ O estoque mínimo deve apoiar alertas de reposição de peças ou produtos.
- ☐ Usuários devem ter permissões compatíveis com suas funções no sistema.

5.7 Segurança, Autenticação e Controle de Acesso

A modelagem prevê separação entre autenticação e dados complementares do usuário. A autenticação pode ser gerenciada pelo serviço `auth.users`, enquanto a tabela `profiles` armazena informações de perfil e `user_roles` define os papéis de acesso.

O controle de acesso deve limitar funcionalidades conforme o papel do usuário. Usuários com perfil de atendente podem registrar vendas, abrir ordens de serviço e cadastrar clientes. Usuários com perfil administrativo podem gerenciar estoque, cadastrar produtos e gerar relatórios.

Além disso, por armazenar dados pessoais de clientes, o sistema deve adotar medidas de proteção compatíveis com a Lei Geral de Proteção de Dados (LGPD), incluindo autenticação segura, restrição de acesso por perfil, uso de conexão criptografada e validação de entradas.

5.8 Considerações sobre PostgreSQL e Supabase

A estrutura apresentada é compatível com banco de dados PostgreSQL, especialmente pela utilização de tipos como `uuid`, `numeric`, `text` e `timestampz`. Também há indicação de integração com Supabase, principalmente pela referência ao schema de autenticação `auth.users` e pelo uso de UUIDs.

No contexto do projeto TrinixTech, a adoção de uma base relacional contribui para a organização das informações e para a confiabilidade dos processos de atendimento técnico, vendas e controle de estoque.

6 TECNOLOGIAS UTILIZADAS

6.1 Front-end

- ☐ HTML5: Estruturação das páginas e conteúdo.
- ☐ CSS3: Estilização visual e responsividade.
- ☐ JavaScript (ES6+): Interatividade e lógicas de interface.
- ☐ Framework moderno (opcional): React ou Next.js para melhor desempenho e modularidade.

6.2 Back-end (Opcional)

- ☐ Node.js: Execução de lógica no servidor.
- ☐ Express.js: Criação de rotas, APIs e integração com formulários.

6.3 Banco de Dados (Opcional)

SQLite ou MongoDB, caso haja área administrativa ou gerenciamento de conteúdo.

6.4 SEO e Monitoramento

- ☐ Google Analytics: Monitoramento de tráfego.
- ☐ Google Search Console: Indexação e análise de presença no Google.
- ☐ Metadados otimizados: Títulos, descrições e URLs amigáveis.

6.5 Integrações

- ☐ Integração com e-mail para formulários de contato.
- ☐ Integração com redes sociais (Instagram, LinkedIn, etc).

6.6 Hospedagem e Deployment

- ☐ Vercel: Ideal para projetos em Next.js.
- ☐ Netlify: Para deploy rápido e simples.
- ☐ GitHub Pages: Para projetos estáticos.
- ☐ Hospedagens tradicionais (Hostinger, Locaweb).

6.7 Ferramentas de Suporte

- ☐ Figma: Prototipação de layout e UX/UI.
- ☐ Git + GitHub: Controle de versão.
- ☐ Visual Studio Code: Ambiente de desenvolvimento.

7 FRONTEND

O presente trabalho descreve o desenvolvimento do front-end do sistema TrinixTech, uma aplicação web voltada para o gerenciamento de uma loja especializada em assistência técnica de eletrônicos. O sistema permite o controle de ordens de serviço, vendas, estoque e cadastro de clientes por meio de uma interface moderna e intuitiva. A aplicação foi construída utilizando tecnologias web atuais, com foco na usabilidade e na organização visual das informações.

Palavras-chave: tecnologia da informação, desenvolvimento web, assistência técnica, sistemas de informação.

7.1 Introdução

Com o crescimento da demanda por serviços de manutenção de dispositivos eletrônicos, torna-se cada vez mais importante que as lojas desse segmento contem com ferramentas digitais para organizar seus processos internos. A ausência de um sistema adequado pode resultar em falhas no controle de ordens de serviço, dificuldades na gestão de estoque e perda de informações sobre clientes.

O TrinixTech surge como uma solução para esse cenário: um sistema web desenvolvido especificamente para auxiliar a gestão de uma loja de assistência técnica de eletrônicos. O foco deste trabalho está no front-end da aplicação, responsável pela interface com o usuário.

7.1.1 Objetivo

O objetivo deste trabalho é documentar o desenvolvimento do front-end do sistema, apresentando as telas desenvolvidas, as funcionalidades implementadas e as tecnologias utilizadas.

7.1.2 Justificativa

A digitalização dos processos em pequenas e médias empresas é um passo fundamental para aumentar a eficiência operacional. Este projeto visa contribuir com essa realidade, oferecendo uma interface clara e funcional para o gerenciamento de uma assistência técnica.

7.2 Tecnologias usadas

O front-end do TrinixTech foi desenvolvido com tecnologias modernas do ecossistema web, priorizando produtividade e qualidade visual. As principais ferramentas utilizadas foram:

- React – biblioteca JavaScript para construção de interfaces dinâmicas e componentizadas.
- TypeScript – superset do JavaScript que adiciona tipagem estática ao código, reduzindo erros em tempo de desenvolvimento.
- Tailwind CSS – framework de estilização utilitário que agiliza a criação do layout responsivo.
- Lovable – plataforma de desenvolvimento assistido por IA utilizada como base para geração e organização dos componentes.
- Supabase – serviço de banco de dados e autenticação utilizado como back-end da aplicação.

7.3 Estrutura da aplicação

A aplicação é dividida em módulos funcionais acessados pelo menu lateral. Cada módulo representa uma área de gestão da loja. A seguir, são descritos os principais módulos implementados.

7.3.1 Dashboard

A tela inicial do sistema apresenta um painel de visão geral com os principais indicadores da loja: ordens de serviço em aberto, total de vendas no mês, quantidade de itens no estoque e alertas de estoque crítico. Também exibe as ordens de serviço mais recentes e os itens com baixo estoque, facilitando o acompanhamento do dia a dia.

7.3.2 Ordens de serviço

Neste módulo, o usuário pode visualizar, criar e acompanhar as ordens de serviço abertas. Cada ordem exibe informações como número de identificação, dispositivo, problema relatado, cliente vinculado, valor e status atual (aberta, em andamento, concluída). É possível alterar o status diretamente na listagem.

7.3.3 Vendas

A tela de vendas lista todas as transações realizadas, com dados de cliente, forma de pagamento, data e valor total. O sistema oferece a opção de registrar uma nova venda com baixa automática no estoque dos produtos vendidos.

7.3.4 Vendas do mês

Este módulo apresenta um relatório mensal de vendas com filtros por mês e ano. As informações exibidas incluem faturamento total, número de vendas, ticket médio e formas de pagamento utilizadas. Também são apresentados gráficos de vendas por dia e detalhamento individual de cada transação.

7.3.5 Estoque

O módulo de estoque permite o gerenciamento das peças e acessórios da loja. Cada item é cadastrado com nome, código SKU, preço e quantidade em estoque. O sistema alerta quando algum item está com estoque abaixo do mínimo configurado.

7.3.6 Clientes

A tela de clientes armazena o cadastro dos clientes da loja, com nome, telefone e e-mail. Os dados podem ser editados ou excluídos conforme necessário, e o cadastro de um novo cliente pode ser feito diretamente nesta tela.

7.4 Interface e usabilidade

A interface do TrinixTech foi desenvolvida com foco na simplicidade e na objetividade. O layout utiliza tema escuro (dark mode), com alto contraste entre os elementos para facilitar a leitura. O menu lateral fixo garante uma navegação rápida entre os módulos, sem necessidade de recarregar a página.

Os cartões de métricas no dashboard utilizam ícones coloridos para identificação visual rápida. As tabelas de dados são padronizadas em todos os módulos, com botões de ação (editar e excluir) ao final de cada linha. Botões de ação principal (como 'Nova OS' e 'Novo item') são destacados em azul ciano, seguindo a identidade visual da marca.

A responsividade da aplicação permite o uso em diferentes tamanhos de tela, tornando-a acessível tanto em computadores quanto em dispositivos menores.

7.5 Considerações

O desenvolvimento do front-end do TrinixTech permitiu a criação de uma interface funcional e organizada para a gestão de uma loja de assistência técnica. As funcionalidades implementadas cobrem as principais necessidades operacionais do negócio: controle de ordens de serviço, vendas, estoque e clientes.

O projeto demonstrou a aplicabilidade das tecnologias web modernas na resolução de problemas reais de pequenas empresas. Como trabalho acadêmico, contribuiu para a consolidação de conhecimentos em desenvolvimento de interfaces, componentização e integração com serviços de back-end.

Como possíveis melhorias futuras, destacam-se a implementação de relatórios exportáveis em PDF, o desenvolvimento de um aplicativo móvel integrado ao sistema e a adição de notificações automáticas para ordens de serviço.

8 BACKEND

Este documento descreve o back-end do sistema TrinixTech, uma aplicação web para gerenciamento de lojas de assistência técnica de eletrônicos. O back-end foi implementado com o Supabase, plataforma que oferece banco de dados relacional (PostgreSQL), autenticação de usuários, controle de acesso por políticas de segurança (Row Level Security) e API gerada automaticamente. São documentados o modelo de dados, as tabelas criadas, as regras de negócio aplicadas e a forma como o front-end consome os serviços.

Palavras-chave: back-end, banco de dados, Supabase, PostgreSQL, API, autenticação.

8.1 Introdução

O back-end de uma aplicação web é responsável por armazenar, processar e disponibilizar os dados que a interface do usuário consome. No contexto do TrinixTech, o back-end precisa suportar operações de leitura e escrita em tempo real, autenticação segura de usuários e regras de acesso que garantam a integridade das informações.

Para atender a esses requisitos de forma prática e dentro do escopo de um projeto acadêmico, foi adotado o Supabase como solução de back-end. Trata-se de uma plataforma de código aberto que oferece banco de dados PostgreSQL gerenciado, autenticação integrada, armazenamento de arquivos e geração automática de APIs REST e em tempo real.

8.1.1 Objetivo

Documentar a estrutura de back end do sistema TrinixTech, descrevendo o banco de dados, as tabelas, os relacionamentos, as políticas de segurança e a integração com o front-end.

8.1.2 Justificativa

A escolha do Supabase como back-end se justifica pela sua facilidade de uso, integração nativa com aplicações React, suporte a autenticação segura e pela

capacidade de escalar conforme a necessidade do projeto, sem exigir a configuração de um servidor dedicado.

8.2 Tecnologias usadas

O back end da TrinixTech foi construído sobre as seguintes tecnologias e serviços:

- Supabase – plataforma de back-end como serviço (BaaS) baseada em PostgreSQL, utilizada para banco de dados, autenticação e API.
- PostgreSQL – sistema de gerenciamento de banco de dados relacional utilizado internamente pelo Supabase para armazenamento e consulta dos dados.
- Row Level Security (RLS) – mecanismo nativo do PostgreSQL para definir políticas de acesso por linha, garantindo que cada usuário visualize apenas seus próprios dados.
- Supabase Auth – módulo de autenticação que gerencia o cadastro e login de usuários via e-mail e senha, com geração de tokens JWT.
- API REST automática – o Supabase gera automaticamente uma API RESTful para cada tabela do banco, consumida pelo front-end via biblioteca supabase-js.
- Realtime – recurso do Supabase que permite a atualização automática dos dados na interface sem necessidade de recarregar a página.

8.3 Modelo do banco de dados

O banco de dados do TrinixTech é composto por cinco tabelas principais, cada uma responsável por um módulo da aplicação. Todas as tabelas possuem uma coluna `user_id` que referencia o usuário autenticado, permitindo que os dados sejam isolados por conta.

8.3.1 Tabela de Clientes

Armazena o cadastro dos clientes da loja. Cada registro é vinculado ao usuário que o criou, garantindo que diferentes contas não acessem os dados umas das outras.

- `id` – identificador único (UUID, gerado automaticamente).

- ☐ user_id – referência ao usuário autenticado (UUID).
- ☐ nome – nome completo do cliente (texto, obrigatório).
- ☐ telefone – número de contato do cliente (texto).
- ☐ email – endereço de e-mail do cliente (texto).
- ☐ created_at – data e hora de criação do registro (timestamp).

8.3.2 Tabela Ordens de Serviço

Registra as ordens de serviço abertas na loja. Cada ordem pode ser vinculada a um cliente cadastrado e possui um status que indica o andamento do reparo.

- ☐ id – identificador único (UUID).
- ☐ user_id – referência ao usuário autenticado (UUID).
- ☐ cliente_id – chave estrangeira para a tabela clientes (UUID, opcional).
- ☐ dispositivo – nome ou modelo do aparelho a ser reparado (texto, obrigatório).
- ☐ problema – descrição do defeito relatado (texto).
- ☐ status – situação atual da ordem: aberta, em_andamento ou concluída (texto).
- ☐ valor – custo do serviço (numérico decimal).
- ☐ created_at – data e hora de abertura da ordem (timestamp).

8.3.3 Tabela Estoque

Controla as peças e acessórios disponíveis na loja. Quando uma venda é registrada, a quantidade em estoque é atualizada automaticamente.

- ☐ id – identificador único (UUID).
- ☐ user_id – referência ao usuário autenticado (UUID).
- ☐ nome – nome do item (texto, obrigatório).
- ☐ sku – código de identificação do produto (texto, opcional).
- ☐ preco – preço unitário do item (numérico decimal).
- ☐ quantidade – quantidade disponível em estoque (inteiro).
- ☐ created_at – data e hora de cadastro do item (timestamp).

8.3.4 Tabela Vendas

Registra todas as vendas realizadas pela loja. Cada venda pode estar associada a um cliente e a uma ou mais ordens de serviço.

- ☐ id – identificador único (UUID).
- ☐ user_id – referência ao usuário autenticado (UUID).
- ☐ cliente_id – chave estrangeira para a tabela clientes (UUID, opcional).
- ☐ forma_pagamento – método utilizado: Pix, dinheiro, cartão, etc. (texto).
- ☐ total – valor total da venda (numérico decimal).
- ☐ created_at – data e hora da venda (timestamp).

8.3.5 Tabela Itens Vendas

Tabela auxiliar que detalha os produtos incluídos em cada venda. Cada registro representa um item vendido, com sua quantidade e preço no momento da venda.

- ☐ id – identificador único (UUID).
- ☐ venda_id – chave estrangeira para a tabela vendas (UUID, obrigatório).
- ☐ estoque_id – chave estrangeira para a tabela estoque (UUID, obrigatório).
- ☐ quantidade – quantidade do item vendido (inteiro).
- ☐ preco_unitario – preço do item no momento da venda (numérico decimal).

8.4 Autenticação e Controle de acesso

O TrinixTech utiliza o módulo Supabase Auth para gerenciar o acesso à aplicação. O fluxo de autenticação funciona da seguinte forma: o usuário informa seu e-mail e senha na tela de login; o Supabase valida as credenciais e retorna um token JWT (JSON Web Token); esse token é armazenado no navegador e enviado em todas as requisições subsequentes, identificando o usuário nas operações de banco de dados.

8.4.1 Autenticação de usuários

O sistema suporta cadastro de novos usuários e recuperação de senha via e-mail, funcionalidades já integradas ao Supabase Auth sem necessidade de implementação adicional.

8.4.2 Row Level Security (RLS)

Para garantir que cada usuário acesse apenas seus próprios dados, todas as tabelas do banco utilizam políticas de Row Level Security (RLS). Essa configuração é feita diretamente no banco de dados e garante isolamento total entre contas diferentes.

A política aplicada em todas as tabelas segue o padrão: operações de leitura, inserção, atualização e exclusão são permitidas somente quando o campo `user_id` da linha corresponde ao identificador do usuário autenticado na sessão atual. Dessa forma, mesmo que dois usuários compartilhem o mesmo banco, nenhum deles consegue visualizar ou modificar os dados do outro.

8.5 Integração com o Front End

O front-end do TrinixTech se comunica com o back-end exclusivamente por meio da biblioteca oficial `supabase-js`. Essa biblioteca abstrai as chamadas HTTP à API REST gerada pelo Supabase, permitindo realizar operações de banco de dados diretamente no código JavaScript/TypeScript.

A conexão é configurada uma única vez, utilizando a URL do projeto e a chave pública (`anon key`) fornecidas pelo Supabase. Todas as requisições são automaticamente autenticadas com o token JWT do usuário logado.

8.5.1 Principais Operações

As operações mais utilizadas na integração são:

- ☐ Listagem de registros – busca todos os registros de uma tabela pertencentes ao usuário autenticado, com suporte a filtros, ordenação e paginação.
- ☐ Inserção – adiciona um novo registro na tabela, com o `user_id` preenchido automaticamente a partir da sessão.
- ☐ Atualização – modifica um registro existente, identificado pelo seu id.
- ☐ Exclusão – remove um registro da tabela com base no id fornecido.
- ☐ Realtime – assinatura de eventos em tempo real para atualizar a interface automaticamente quando um dado é alterado.

8.5.2 Baixa automática de estoque

Uma das regras de negócio implementadas no back-end é a baixa automática de estoque ao registrar uma venda. Ao inserir uma venda com itens, o sistema subtrai a quantidade vendida do campo quantidade na tabela estoque para cada item incluído na transação. Essa lógica é executada no front-end por meio de chamadas sequenciais à API, garantindo consistência entre os módulos de vendas e estoque.

8.6 Considerações

O back-end do TrinixTech, implementado com o Supabase, atendeu de forma satisfatória às necessidades do sistema. A plataforma permitiu configurar o banco de dados, a autenticação e as regras de acesso sem a necessidade de desenvolver e manter um servidor próprio, o que foi essencial para viabilizar o projeto no prazo acadêmico.

O uso de Row Level Security garantiu a segurança e o isolamento dos dados por usuário, enquanto a API automática do Supabase simplificou significativamente a integração com o front-end. O módulo Realtime, por sua vez, possibilitou que as atualizações de dados fossem refletidas na interface sem a necessidade de recarregar a página.

Como melhorias futuras para o back-end, destacam-se o uso de triggers para automatizar ações como o envio de notificações ao alterar o status de uma O.S.

9 DOCUMENTAÇÃO DAS SPRINTS

9.1 Sprint 1 – Planejamento e Design Inicial

Durante a primeira sprint foi realizado o levantamento de requisitos do sistema, definição das funcionalidades principais e estrutura inicial do projeto. Também foram desenvolvidos os primeiros protótipos de telas e organização do fluxo de navegação do sistema. Foram utilizadas ferramentas de prototipação, documentação técnica e definição da arquitetura inicial do banco de dados.

Principais atividades:

- ☐ Levantamento de requisitos
- ☐ Estruturação do banco de dados
- ☐ Criação de wireframes
- ☐ Definição das funcionalidades do sistema
- ☐ Planejamento das telas

9.2 Sprint 2 – Design Final e Documentação

Na segunda sprint foi realizada a finalização do design da aplicação e a documentação detalhada do sistema. Foram ajustadas interfaces, definidos padrões visuais e organizadas as regras de negócio da aplicação. Também foram documentados os requisitos funcionais e não funcionais do sistema.

Principais atividades:

- ☐ Ajustes de interface
- ☐ Padronização visual
- ☐ Documentação técnica
- ☐ Definição das regras de negócio
- ☐ Organização da estrutura do projeto

9.3 Sprint 3 – Desenvolvimento do Sistema

Na terceira sprint iniciou-se o desenvolvimento do sistema, incluindo criação das funcionalidades principais relacionadas ao controle de estoque, cadastro de clientes e ordens de serviço. Também foi realizada a integração com o banco de dados e implementação da lógica de funcionamento do sistema.

Principais atividades:

- ☐ Desenvolvimento do backend
- ☐ Cadastro de clientes
- ☐ Controle de estoque
- ☐ Criação das ordens de serviço
- ☐ Integração com banco de dados

9.4 Sprint 4 – Finalização e Testes

Na quarta sprint foram concluídas as funcionalidades restantes, realizados testes de funcionamento e correções de erros identificados durante o desenvolvimento. Também foram implementados relatórios, melhorias de desempenho e validações de segurança do sistema.

Principais atividades:

- ☐ Finalização das funcionalidades
- ☐ Testes do sistema
- ☐ Correção de bugs
- ☐ Implementação de relatórios
- ☐ Ajustes de desempenho e segurança

10 CONCLUSÃO

O desenvolvimento do Sistema de Gestão TrinixTech permitiu pôr em prática os conhecimentos adquiridos durante o curso, englobando áreas como análise de sistemas, modelagem UML, banco de dados e desenvolvimento web para empresas.

Ao longo do projeto, foi possível identificar a importância da tecnologia na organização e otimização dos processos empresariais. A solução desenvolvida buscou atender às necessidades propostas, oferecendo uma ferramenta capaz de auxiliar no gerenciamento de informações, produtos, serviços e vendas de forma mais eficiente.

Além dos resultados obtidos com o sistema, o projeto contribuiu para o desenvolvimento das habilidades técnicas e do trabalho em equipe, proporcionando uma experiência próxima à realidade do mercado de tecnologia. A parceria com a empresa também permitiu compreender melhor os desafios enfrentados pelas organizações e a importância de desenvolver soluções alinhadas às suas necessidades.

Dessa forma, conclui-se que os objetivos definidos para o projeto foram alcançados, resultando em uma solução funcional e contribuindo para o aprendizado e crescimento acadêmico dos integrantes do grupo.

11 REFERÊNCIAS

BOOCH, Grady; RUMBAUGH, James; JACOBSON, Ivar. UML: Guia do Usuário. 2. ed. Rio de Janeiro: Elsevier, 2012.

PRESSMAN, Roger S.; MAXIM, Bruce R. Engenharia de Software: uma abordagem profissional. 9. ed. Porto Alegre: AMGH, 2021.

SOMMERVILLE, Ian. Engenharia de Software. 10. ed. São Paulo: Pearson, 2019.

MDN WEB DOCS. Web Docs. Disponível em: <https://developer.mozilla.org/>. Acesso em: 13 jun. 2026.

W3SCHOOLS. W3Schools Online Web Tutorials. Disponível em: <https://www.w3schools.com/>. Acesso em: 13 jun. 2026.

REACT. Documentação oficial do React. Disponível em: <https://react.dev>. Acesso em: jun. 2026.

TAILWIND CSS. Documentação oficial do Tailwind CSS. Disponível em: <https://tailwindcss.com>. Acesso em: jun. 2026.

SUPABASE. Documentação oficial do Supabase. Disponível em: <https://supabase.com/docs>. Acesso em: jun. 2026.

TYPESCRIPT. Documentação oficial do TypeScript. Disponível em: <https://www.typescriptlang.org>. Acesso em: jun. 2026.

JSON WEB TOKEN. Introduction to JSON Web Tokens. Disponível em: <https://jwt.io/introduction>. Acesso em: jun. 2026.

POSTGRESQL. Documentação oficial do PostgreSQL. Disponível em: <https://www.postgresql.org/docs>. Acesso em: jun. 2026.

SUPABASE. Row Level Security. Disponível em: <https://supabase.com/docs/guides/database/postgres/row-level-security>. Acesso em: jun. 2026.

SUPABASE. Supabase Auth. Disponível em: <https://supabase.com/docs/guides/auth>. Acesso em: jun. 2026.